

visual-book-md

本書はバージョン管理システム Git の基本的な使い方を解説します。初めて Git を使う方から、チーム開発に活用したい方まで幅広く対応しています。

第 1 章 Git とは何か

本章では、バージョン管理システムである Git の基本概念と、本書全体の構成について説明する。

Git は、ファイルの変更履歴を記録・管理するための分散型バージョン管理システムです。¹ プログラムの変更内容をコミットという単位で保存し、過去の任意の時点の状態に戻したり、複数の作業を並行して進めたりすることができます。

本書の構成

本書は以下の 4 章から構成されます。

- ・ 第 1 章：Git とは何か
 - ・ バージョン管理の基本概念
 - ・ Git を使う理由
- ・ 第 2 章：基本的な使い方
 - ・ リポジトリの作成とコミット
 - ・ 設定ファイルの記述方法
- ・ 第 3 章：ブランチを用いた作業
 - ・ ブランチの作成・統合
- ・ 第 4 章：リモートリポジトリとの連携
 - ・ GitHub を用いたチーム開発

Git を使う理由

ソフトウェア開発では、ファイルを少しずつ変更しながら作業を進めます。Git を使うと、変更の単位をコミットとして記録し、作業の流れを明確に残すことができます

¹ バージョン管理システムとは、ファイルの変更履歴を記録・追跡するためのソフトウェアです。Git 以外にも Subversion や Mercurial などがあります。

² macOS・Linux では ~/.gitconfig, Windows では %USERPROFILE%\.gitconfig に保存されます。

す。また、ブランチ機能を活用することで、複数の作業を独立して進めることができます。

第 2 章 基本的な使い方

本章では、Git リポジトリの作成からコミットの記録まで、基本的な操作の流れを説明する。設定ファイルの書き方についても解説する。

Git の設定

Git を初めて使う際には、ユーザ名とメールアドレスを設定します。これらの情報は、コミットに記録される著者情報として使われます。設定はホームディレクトリの² .gitconfig ファイルに保存されます。

ソースコード 1: .gitconfig

```
1 [user]
2   name  = Your Name
3   email = you@example.com
4
5 [core]
6   editor = vim
7
8 [alias]
9   st = status
10  co = checkout
11  br = branch
12  lg = log --oneline --graph
```

リポジトリの作成

Git で管理する作業場所をリポジトリと呼びます。既

存のディレクトリを Git で管理するには、次のコマンドを実行します。

```
git init
```

このコマンドを実行すると、現在のディレクトリに Git の管理情報が格納された `.git` ディレクトリが作成されます。

変更の記録

Git では、変更したファイルをすぐに履歴として記録するのではなく、まずステージング領域に追加します。その後、コミットによって履歴として保存します。

```
git add main.ts
git commit -m "Add main script"
```

以下に、頻繁に使用する主な Git コマンドをまとめます。

コマンド	説明
<code>git status</code>	作業ツリーの状態を確認する
<code>git add <file></code>	変更をステージングに追加する
<code>git commit</code>	ステージングの内容を履歴に保存する
<code>git log</code>	コミット履歴を一覧表示する
<code>git diff</code>	作業ツリーとステージングの差分を確認する

コミットメッセージ

コミットメッセージには、その変更で何を行ったのかを短く具体的に書きます。たとえば「Fix bug」だけではなく、「Fix path handling in config loader」のように対象と内容が分かる表現にするとよいです。

第 3 章 ブランチを用いた作業



図 1: ブランチを用いた開発フロー

ブランチの役割

ブランチは、作業の流れを分岐させるための仕組みです。上図に示すように、新しい機能を追加するときや既存の処理を修正するときにブランチを作成すると、main ブランチを安定した状態に保ったまま作業できます。

ブランチの作成と移動

新しいブランチを作成してそのブランチに移動するには、次のコマンドを使います。

```
git switch -c feature/login
```

この例では、`feature/login` という名前のブランチを作成しています。ブランチ名には、作業内容が分かる名前を付けると、後から見たときに意図を理解しやすくなります。

差分の確認

作業中の変更内容を確認するには、次のコマンドを使います。差分を確認してからコミットすることで、意図しない変更を履歴に含めることを防げます。

```
git diff
```

ブランチの統合

作業ブランチでの変更を main ブランチに取り込むには、main ブランチに移動してから `merge` を実行します。

```
git switch main
git merge feature/login
```

マージによって、作業ブランチで行った変更が main ブランチに統合されます。競合が発生した場合は、Git が自動で判断できなかった箇所を手動で修正する必要があります。

第4章 リモートリポジトリとの連携

本章では、GitHub などのリモートリポジトリと連携して、チームで開発を進める方法を説明する。

リモートリポジトリとは

リモートリポジトリは、ネットワーク上に置かれた Git リポジトリです。GitHub や GitLab などのサービスを利用すると、ローカル環境で作成した履歴を共有し、複数人で同じプロジェクトを開発できます。

リモートの登録と変更の送信

ローカルリポジトリにリモートリポジトリを登録して、コミットを送信するまでの流れを示します。

```
git remote add origin
git@github.com:example/project.git
git push -u origin main
```

初回の push では、`-u` オプションを付けることで、ローカルの main ブランチとリモートの main ブランチを対応付けます。次回以降は、単に `git push` と入力するだけで送信できます。

変更の取得

リモートリポジトリ上の変更を取得して現在のブランチに取り込むには、`pull` を使います。

```
git pull
```

good commit = small change + clear message

まとめ

Git の基本は、変更を確認し、必要なものをステージングし、意味のある単位でコミットすることです。ブランチを使って作業を分け、リモートリポジトリと連携することで、個人開発からチーム開発まで幅広い場面に対応できます。